



Effects with User Interactions



Learning Objectives

- Build a component with an effect hook that watches for changes to a given value and triggers the effect
- Build a component that is passed data via props with an effect hook that watches for changes to a props value



useEffect working with state

Here we have an HTTP request that is triggered from a user interaction

Q: What type of user interaction might trigger the effect?

```
function App() {  
  const [data, setData] = useState(null)  
  const [searchQuery, setSearchQuery] = useState("")  
  
  useEffect(() => {  
    fetch(`https://example.com/?search=${searchQuery}`)  
      .then(res => res.json())  
      .then(data => setData(data))  
  }, [searchQuery])  
  ...  
}
```



useEffect working with state

Potentially a form submission event

```
function handleSubmit(event) {  
  event.preventDefault()  
  
  setSearchQuery(searchInput)  
}
```



useEffect working with state

When a user clicks submit it will run the `setSearchQuery` function and set state....

```
function handleSubmit(event) {  
  event.preventDefault()  
  
  setSearchQuery(searchInput)  
}
```



useEffect working with state

So now `searchQuery` has the user's input stored in it and the component re-renders

Our `useEffect` has instructions to keep track of any changes to `searchQuery` by using the dependency array

This means anytime `searchQuery` is changed run the code inside the `useEffect`

Let's see that in action!

```
function App() {  
  const [data, setData] = useState(null)  
  const [searchQuery, setSearchQuery] = useState("")  
  
  useEffect(() => {  
    fetch(`https://example.com/?search=${searchQuery}`)  
      .then(res => res.json())  
      .then(data => setData(data))  
  }, [searchQuery])  
  
  ...  
}
```



Time for some live coding

Let's implement useEffect from state [here](#)

- Add code to App.js
- Add code to SearchForm.js
- The API to use in the examples linked here is:
<https://swapi.dev/> as it has changed

Complete version [here](#)



Time for some practice

<https://github.com/boolean-uk/react-use-effect-dependencies-practice-1>

Complete version [here](#)



useEffect working with props

As well as running code when a state value changes, we can also use **useEffect** to run code when the value of a **prop passed in to a component changes**.

A common use case for this is when data from an initial fetch request updates a component, and then that component wants to fetch some additional data.

We can fetch that **additional data** within our components with **useEffect**

```
function Component(props) {  
  const dataWithUrl = props.dataWithUrl  
  
  const [data, setData] = useState(null)  
  
  useEffect(() => {  
    fetch(dataWithUrl.url)  
      .then(res => res.json())  
      .then(data => setData(data))  
  }, [dataWithUrl])  
  ...  
}
```



useEffect working with props

Our **useEffect** has instructions to keep track of any changes to `dataWithUrl` from props by using the dependency array

This means anytime `dataWithUrl` is changed, run the code inside the **useEffect**

Let's see that in action!

```
function Component(props) {  
  const dataWithUrl = props.dataWithUrl  
  
  const [data, setData] = useState(null)  
  
  useEffect(() => {  
    fetch(dataWithUrl.url)  
      .then(res => res.json())  
      .then(data => setData(data))  
  }, [dataWithUrl])  
  ...  
}
```



Time for some live coding

Let's implement `useEffect` from props [here](#)

- Add to `PeopleListItem` component

Complete version [here](#)



Time for some practice

<https://github.com/boolean-uk/react-use-effect-dependencies-practice-2>

Your instructor will demo the first part.